

Table of Contents

System Overview	2
Technology Stack	2
Data Models	3
Core Models and Relationships	3
User Role System	4
API Architecture	5
Authentication Endpoints	5
Dashboard Endpoint	5
Core Business Endpoints	6
Frontend Architecture	6
MCLUB Staff Routes	6
SME Owner Routes	7
Agent Routes	7
End User Routes	8
Core Business Flows	8
Settlement (Revenue Sharing) Flow	8
Event Lifecycle Flow	9
Membership Tier Progression	10
Infrastructure and Deployment	10
Deployment Architecture	10
Access Information	11
Data Model Relationships	11

System Overview

MCLUB CRM (Customer Relationship Management) system is a multi-role family office CRM platform designed and developed for Park Zeman Group. The system integrates client management, order settlement (also known as revenue sharing or commission splitting), commission tracking, and event management into a unified digital platform. It serves as the operational backbone for the MCLUB brand, enabling efficient coordination among staff, SME partners, agents, and end users.

The platform is built on a modern technology stack featuring Next.js 16 with React 19 on the frontend, Prisma ORM with SQLite on the backend, and Bun as the JavaScript runtime. The system is deployed behind a Caddy reverse proxy and managed via PM2 process manager, ensuring high availability and smooth operation. The entire application follows a full-stack TypeScript architecture, enabling type safety from database schema to UI components.

At its core, the MCLUB CRM implements a sophisticated multi-tenant role-based access control system with four distinct user roles: MCLUB Staff (platform administrators), SME Owners (business partners who list products), Agents (referral partners who earn commissions), and End Users (customers who purchase products and attend events). Each role has a tailored dashboard and feature set, ensuring that users only see relevant information and can perform actions appropriate to their permission level.

Technology Stack

The MCLUB CRM system leverages a carefully selected technology stack that balances developer productivity, performance, and maintainability. Each technology choice was made to support the specific requirements of a multi-role CRM system with real-time data operations, complex business logic for revenue sharing, and a responsive user interface.

Layer	Technology	Purpose
Frontend Framework	Next.js 16 + React 19	Server-side rendering, API routes, routing
UI Components	shadcn/ui + Radix UI	Accessible, customizable component library
Styling	Tailwind CSS 4	Utility-first CSS framework for rapid UI development
State Management	Zustand	Lightweight client-side state management

Data Visualization	Recharts	Chart library for dashboards and reports
API Layer	Next.js App Router	RESTful API endpoints with route handlers
ORM	Prisma v6	Type-safe database access and schema management
Database	SQLite	Lightweight embedded database for data persistence
Runtime	Bun	High-performance JavaScript/TypeScript runtime
Process Manager	PM2	Production process management and monitoring
Reverse Proxy	Caddy (port 81)	Automatic HTTPS, reverse proxy to port 3000
Icons	Lucide Icons	Consistent icon set across the application

Table 1: Technology Stack Overview

The choice of Next.js 16 as the full-stack framework provides significant advantages: server-side rendering for fast initial page loads, built-in API routes that eliminate the need for a separate backend service, and the App Router for intuitive file-based routing. Combined with Prisma ORM, the system achieves end-to-end type safety from database queries to React component props, reducing runtime errors and improving developer confidence during feature development.

Data Models

The MCLUB CRM system is built on nine core data models defined in the Prisma schema. These models form the foundation of the entire application, representing the key business entities and their relationships. The data model design follows a relational approach with clear foreign key references, ensuring data integrity and enabling complex queries across related entities.

Core Models and Relationships

Model	Description	Key Relationships
-------	-------------	-------------------

User	System users with 4 role types	User creates Clients, User places Orders
Client	SME/Agent client records	Client has TimelineEvents, belongs to User
Product	Products listed by SME Owners	Product appears in Orders
Order	Purchase orders with settlement	Order links to User, Product, and Client
Commission	Agent commission tracking	Commission belongs to Agent (User)
ClubEvent	Events with lifecycle management	Event has Attendees, Tasks, BudgetItems
EventAttendee	RSVP and check-in records	Attendee belongs to ClubEvent and User
EventTask	Event task management	Task belongs to ClubEvent
EventBudgetItem	Event budget line items	BudgetItem belongs to ClubEvent
TimelineEvent	Client activity timeline log	TimelineEvent belongs to Client

Table 2: Core Data Models

User Role System

The User model implements a role-based access control system with four distinct roles, each with specific capabilities and access permissions. This design ensures that every user interacts with the system through a role-appropriate lens, preventing unauthorized access while maintaining operational efficiency.

Role	Email	Primary Capabilities
MCLUB_STAFF	kenneth@parkzeman.com	Full admin: clients, orders, products, commissions, events, settlement
SME_OWNER	calvin@mclub.com	Product listing, income tracking, client management
AGENT	agent@mclub.com	Client referrals, commission management
END_USER	user@mclub.com	Purchase history, event registration

Table 3: User Roles and Permissions

Each role has a dedicated dashboard that surfaces the most relevant metrics and actions. MCLUB Staff users see a comprehensive overview of all system activity, including pending settlements, event RSVPs, and commission payouts. SME Owners focus on product performance and revenue metrics, while Agents track their referral pipeline and earned commissions. End Users have a simplified view showing their purchase history and upcoming events.

API Architecture

The MCLUB CRM backend is implemented as a set of Next.js App Router API route handlers. Each route follows RESTful conventions with role-based access control middleware that validates the authenticated user before processing any request. The API layer handles all business logic, data validation, and database operations through Prisma ORM.

Authentication Endpoints

Endpoint	Method	Description
/api/auth/login	POST	User authentication with email and password, returns JWT token
/api/auth/profile	GET	Retrieve current authenticated user profile and role

Table 4: Authentication API

Dashboard Endpoint

The dashboard API endpoint provides role-specific aggregated data for each user type. When a user accesses their dashboard, the system queries and computes relevant metrics based on the user role, returning tailored statistics such as total clients, revenue figures, commission totals, or upcoming events. This design eliminates the need for multiple dashboard API calls, reducing frontend complexity and improving page load performance.

Endpoint	Method	Description
/api/dashboard/[role]	GET	Returns role-based dashboard data (aggregated metrics)

Table 5: Dashboard API

Core Business Endpoints

Endpoint	Methods	Key Features
/api/clients	GET, POST, PUT, DELETE	CRUD for client records with membership tier tracking
/api/orders	GET, POST, PUT, DELETE	CRUD for orders, includes settlement (revenue sharing) operation
/api/products	GET, POST, PUT, DELETE	CRUD for product catalog managed by SME Owners
/api/commissions	GET, POST, PUT, DELETE	CRUD for agent commission records and payout tracking
/api/events	GET, POST, PUT, DELETE	CRUD plus RSVP, check-in, tasks, and budget management

Table 6: Core Business API Endpoints

The API layer implements comprehensive error handling with consistent response formats. All mutation endpoints validate input data against schema constraints before processing, and the system maintains an audit trail through the TimelineEvent model, which automatically records significant client interactions. The settlement operation on the Orders endpoint is particularly critical, as it triggers the revenue distribution logic that splits order payments among agents, SME owners, and the MCLUB platform according to predefined percentage allocations.

Frontend Architecture

The MCLUB CRM frontend follows a role-based routing architecture where each user type has its own route prefix and dedicated set of pages. This approach provides clear separation of concerns and ensures that the UI can be tailored precisely to each role workflow. The application uses Next.js App Router with layout components that enforce role-specific navigation and sidebar menus.

MCLUB Staff Routes

Route	Description
/mclub-admin/dashboard	Admin dashboard with system-wide metrics and KPIs
/mclub-admin/clients	Client management with search, filter, and CRUD operations
/mclub-admin/orders	Order management with settlement workflow
/mclub-admin/products	Product catalog management across all SME partners
/mclub-admin/commissions	Commission tracking and payout management
/mclub-admin/events/*	Event management with lifecycle controls

Table 7: MCLUB Staff Frontend Routes

SME Owner Routes

Route	Description
/sme-owner/dashboard	SME dashboard with revenue and product performance metrics
/sme-owner/products	Product management for SME listings
/sme-owner/orders	Order tracking for SME products

Table 8: SME Owner Frontend Routes

Agent Routes

Route	Description
/agent/dashboard	Agent dashboard with referral pipeline and commission summary
/agent/clients	Client referral management
/agent/commissions	Commission view and payout history

Table 9: Agent Frontend Routes

End User Routes

Route	Description
/user/dashboard	User dashboard with purchase history and event calendar
/user/orders	Purchase history and order details
/user/events	Event listing with RSVP functionality

Table 10: End User Frontend Routes

Core Business Flows

The MCLUB CRM system implements several interconnected business processes that form the operational backbone of the platform. Understanding these flows is essential for grasping how the system creates value for all stakeholders and ensures transparent, automated revenue distribution.

Settlement (Revenue Sharing) Flow

The settlement flow is the most critical business process in the MCLUB CRM system. It implements an automated revenue sharing mechanism that distributes order payments among three parties: the Agent who referred the client, the SME Owner who provides the product, and the MCLUB platform itself. This multi-party distribution ensures that every transaction is transparently tracked and that all stakeholders receive their entitled share without manual intervention.

The flow begins when an Agent refers a client to the platform. The client then purchases a product listed by an SME Owner, generating an Order record. Once the order is confirmed, the settlement process is triggered, which calculates the distribution based on predefined percentage allocations. The Agent receives their commission percentage, the SME Owner receives their revenue share, and the MCLUB platform retains its service fee. All distributions are recorded in the Commission model for audit and reporting purposes.

Step	Actor	Action	System Effect
1	Agent	Refers a client to MCLUB	Client record linked to Agent

2	End User	Purchases a product	Order record created with status Pending
3	MCLUB Staff	Confirms the order	Order status changes to Confirmed
4	System	Executes settlement	Revenue split calculated and recorded
5	System	Distributes shares	Agent commission, SME revenue, MCLUB fee allocated

Table 11: Settlement Flow Steps

Event Lifecycle Flow

The event management module implements a complete lifecycle for club events, from initial drafting through to completion. Each event progresses through five distinct states, with specific actions available at each stage. This structured approach ensures that events are properly planned, promoted, and executed, with full traceability of attendee engagement.

State	Description	Available Actions
DRAFT	Event is being planned, not visible to users	Edit details, add tasks and budget items
PUBLISHED	Event is live and visible to all users	Users can view event details and RSVP
RSVP	Registration phase is active	Attendees confirm attendance, track RSVP count
CHECKIN	Event is happening, check-in is active	Staff can check in attendees, track attendance
COMPLETE D	Event has concluded	View attendance report, budget actuals, feedback

Table 12: Event Lifecycle States

The event lifecycle is managed through dedicated API endpoints that enforce state transition rules. For example, an event cannot be moved to the CHECKIN state without first passing through the PUBLISHED and RSVP states. Similarly, budget items and tasks can only be modified while the event is in DRAFT status, preventing unauthorized changes after publication. The EventAttendee model tracks both RSVP and check-in timestamps, providing a complete audit trail of attendee engagement.

Membership Tier Progression

The MCLUB CRM implements a membership tier system that rewards client loyalty and engagement. Clients progress through a series of membership levels, each offering enhanced benefits and privileges. The progression from Plan A through Plan C to Full Member status is tracked in the Client model, with the system automatically updating tier status based on predefined criteria such as purchase volume, event attendance, and engagement duration.

Tier	Level	Typical Qualifications
Plan A	Entry Level	New clients who have completed initial registration
Plan B	Intermediate	Clients with qualifying purchase history and engagement
Plan C	Advanced	Long-term clients with sustained activity and referrals
Full Member	Premium	Top-tier clients with significant contribution and loyalty

Table 13: Membership Tier Progression

Infrastructure and Deployment

The MCLUB CRM system is deployed on a streamlined infrastructure stack designed for reliability, ease of maintenance, and cost efficiency. The deployment architecture follows a single-server model with a reverse proxy, which is appropriate for the current scale of operations and can be extended to a multi-server architecture as the platform grows.

Deployment Architecture

The production deployment follows a straightforward request flow: incoming HTTPS requests from the internet are received by Caddy on port 81, which handles SSL termination and reverse proxies requests to the Next.js application running on port 3000. The Next.js application processes requests through its API routes and server-side rendering pipeline, communicating with the SQLite database for data persistence. The entire application process is managed by PM2, which provides automatic restarts, log management, and monitoring capabilities.

Component	Technology	Configuration
-----------	------------	---------------

Reverse Proxy	Caddy	Port 81, SSL termination, proxy to localhost:3000
Application Server	Next.js	Port 3000, SSR + API routes
Runtime	Bun	High-performance JS/TS runtime
Process Manager	PM2	Auto-restart, log management, monitoring
Database	SQLite	Embedded database, file-based storage
Version Control	GitHub	Repository: mypathtravel101-cyber/mclub

Table 14: Infrastructure Components

Access Information

Item	Value
Preview URL	https://preview-chat-cfbf9474-2db8-4ba4-8247-31eed109e08e.space-z.ai/
GitHub Repository	https://github.com/mypathtravel101-cyber/mclub
Project Directory	/home/z/my-project/
Default Password	demo123 (all demo accounts)

Table 15: Access Information

The single-server architecture provides simplicity in deployment and maintenance while maintaining sufficient performance for the current user base. The SQLite database eliminates the need for a separate database server, reducing operational overhead. PM2 ensures that the application remains available through automatic process restarts in case of failures, and its monitoring capabilities allow operators to track memory usage, CPU utilization, and request handling in real time.

Data Model Relationships

The following table summarizes the key relationships between data models in the MCLUB CRM system. Understanding these relationships is essential for developers working on the codebase, as they define how data flows through the system and which models must be queried together for common operations.

Source Model	Relationship	Target Model	Cardinality
User	creates	Client	One-to-Many
User	places	Order	One-to-Many
Product	included in	Order	One-to-Many
Agent (User)	earns	Commission	One-to-Many
ClubEvent	has	EventAttendee	One-to-Many
ClubEvent	has	EventTask	One-to-Many
ClubEvent	has	EventBudgetItem	One-to-Many
Client	has	TimelineEvent	One-to-Many

Table 16: Data Model Relationships

The relationship structure reveals that the User model is central to the system, serving as the parent entity for Clients, Orders, and Commissions through different role-based access patterns. The ClubEvent model is another hub, aggregating attendee records, task assignments, and budget line items. The TimelineEvent model provides a chronological activity log for each client, enabling staff and agents to track all interactions and maintain a complete client history.